

StockKeep

Shop Inventory & Stock Tracking System

Testing Report

Testing_Report.pdf — Standalone Testing Document

Student Name:	[STUDENT NAME — TO BE PROVIDED]
Student ID:	[STUDENT ID — TO BE PROVIDED]
Project Title:	StockKeep — Shop Inventory & Stock Tracking System
Institution:	[INSTITUTION NAME — TO BE PROVIDED]
Department:	[DEPARTMENT NAME — TO BE PROVIDED]
Module / Course:	[MODULE / COURSE CODE — TO BE PROVIDED]
Supervisor:	[SUPERVISOR NAME — TO BE PROVIDED]
Date:	13 August 2026

Tests TC-API-01 through TC-API-26 in this report were executed against a live local instance of the application (`npm run dev`, SQLite dev database) and the results below are the actual, unedited HTTP responses captured at execution time. Raw evidence logs are provided in `Supporting_Files/Test_Evidence/`. Any test that was not executed is explicitly labelled “*Not verified — execution required by student*”; no execution result has been invented.

Contents

1	Testing Strategy	2
2	Testing Scope	3
3	Test Environment	4
4	Functional Testing (Executed)	5
5	Unit Testing	9
6	Integration Testing	10
7	System Testing	11
8	User Acceptance Testing (UAT)	12
9	Security Testing	13
10	Usability Testing	14
11	Performance Testing	15
12	Defects	16
13	Regression Testing	21
14	Testing Summary	22
15	Requirements-to-Test Traceability	23

1. Testing Strategy

Testing for this submission combines three tiers of confidence, and each result in this report is labelled with which tier produced it:

1. **Executed (live evidence).** 26 black-box API test cases (TC-API-01 to TC-API-26) were run against a live local instance of the application using `curl`, with real HTTP status codes and response bodies captured verbatim. This is genuine execution evidence, not a prediction.
2. **Static (code review).** Where live execution was impractical in this environment (e.g., rendered UI behaviour, cross-browser layout), the underlying implementation was read and cross-checked against the requirement, but the claim is explicitly labelled as code-review-only, not as a pass from running the system.
3. **Not verified.** Anything neither executed nor conclusively determinable from source (performance under load, usability with real users, cross-device layout, production-build behaviour) is marked “**Not verified — execution required by student**” rather than assumed.

No automated test suite (unit, integration, or end-to-end) exists in the repository — see Testing Scope below and `Technical_Debt_Plan.pdf` (TD-07). This report is therefore a manual/black-box testing pass intended to establish a baseline, not a report of an existing CI test suite.

2. Testing Scope

In scope: authentication and session handling, item CRUD, stock adjustment business rules, sales recording and stock deduction, supplier CRUD, reports/analytics read endpoints, settings read, AI route fallback behaviour, the unauthenticated-seed security gap, and basic brute-force/header checks.

Out of scope for this pass: rendered UI/visual testing (no browser automation available in this environment — see `FINAL_SUBMISSION_READINESS_REPORT.md`), load/performance testing, cross-browser/cross-device testing, and testing of a production (`next build && next start`) deployment — all executed tests ran against `next dev`.

3. Test Environment

Item	Value
OS	Windows 11 Pro
Runtime	Node.js (via <code>npm run dev</code> , Next.js 16.3.0, Webpack dev server)
Application mode	Development (<code>next dev</code>) — not production build; see caveat in Section 12
Database	SQLite, <code>prisma/dev.db</code> , seeded via <code>ensureSampleDataSeeded()</code>
Server address	<code>http://localhost:3001</code> (port 3000 was occupied, Next.js auto-selected 3001)
HTTP client	<code>curl</code> (command-line), scripted for repeatability
AI configuration	<code>GEMINI_API_KEY</code> present in <code>.env.local</code> (value not disclosed in this document); live Gemini calls were attempted and their real outcome is reported in Section 6
Test data	Live sample dataset from <code>ensureSampleDataSeeded()</code> plus test fixtures created during this run (<code>QA Test Widget</code> , <code>QA Test Supplier</code>)
Date/time of execution	2026-08-13, 13:17–13:23 UTC (see timestamps in <code>Supporting_Files/Test_Evidence/api_test_evidence.log</code>)

4. Functional Testing (Executed)

All rows below were executed live; **Actual Result** is the real, captured server response. Full request/response evidence is in `Supporting_Files/Test_Evidence/api_test_evidence.log`.

Test ID	Req. ID	Test Case	Steps	Expected Result	Actual Result	Status
TC-API-01	FR-03	Unauthenticated page load is gated	GET / with no cookie	Redirect to /login	307 redirect to /login	PASS
TC-API-02	FR-03	Unauthenticated API call is gated	GET /api/items with no cookie	401	401	PASS
TC-API-03	FR-01	Wrong passcode rejected	POST /api/auth/login {"passcode": "0000"}	401, error message	401, {"error": "Incorrect passcode, try again"}	PASS
TC-API-04	FR-01, FR-02	Correct passcode logs in	POST /api/auth/login {"passcode": "1234"}	200, session cookie set	200, stockkeep_auth cookie set (confirmed HttpOnly, SameSite=Lax, Max-Age=604800 — see TC-API-26)	PASS
TC-API-05	FR-08, FR-09	Authenticated read succeeds	GET /api/items with valid cookie	200, item list with supplier/movements	200, full item list returned	PASS
TC-API-06	FR-07	Negative quantity rejected	POST /api/items with quantity:-5	400 with validation message	500, empty body	FAIL — DEF-01
TC-API-07	FR-06	Valid item creation	POST /api/items with valid body	201, created item incl. auto SKU	201, sku: "SKU-013"	PASS

Test ID	Req. ID	Test Case	Steps	Expected Result	Actual Result	Status
TC-API-08	FR-08	Search returns created item	GET <code>/api/items?search=QA</code> Test Widget	200, item present	200, item present with 1 IN movement	PASS
TC-API-09	FR-12	Stock-Out beyond stock rejected	POST <code>/api/items/35/adjust</code> { <code>"type": "OUT", "quantity": 9999</code> }	400, <code>5/adjust</code> {"type": "OUT", "quantity": 9999}	400, "Cannot 9999 units. Current stock is 20..."	PASS
TC-API-10	FR-11, FR-13	Valid Stock-Out succeeds	<code>type: "OUT", quantity: 5</code> on a 20-unit item	200, quantity → 15	200, quantity 15	PASS
TC-API-11	FR-11, FR-13	Valid Stock-In succeeds	<code>type: "IN", quantity: 3</code>	200, quantity → 18	200, quantity 18	PASS
TC-API-12	FR-11	Zero quantity rejected	<code>type: "IN", quantity: 0</code>	400 with validation message	500, empty body	FAIL — DEF-01
TC-API-13	FR-15	Valid sale recorded	POST <code>/api/sales</code> qty 2 of 18 in stock	200, stock → 16, sale total computed	200, quantity 16, total: 25	PASS
TC-API-14	FR-15	Oversell rejected	Sale qty 9999 of 16 in stock	400, insufficient-stock message	400, "Insufficient stock! Requested 9999..."	PASS
TC-API-15	FR-16	Supplier requires name	POST <code>/api/suppliers</code> { <code>"name": ""</code> }	400	400, "Supplier name is required"	PASS
TC-API-16	FR-16	Valid supplier created	Valid name + email	201	201	PASS
TC-API-17	FR-17	Reports summary returns data	GET <code>/api/reports/summary</code>	200, <code>summary</code> breakdown	200, full category breakdown returned	PASS

Test ID	Req. ID	Test Case	Steps	Expected Result	Actual Result	Status
TC-API-18	FR-05	Analytics returns dashboard data	GET /api/analytics	200, KPIs 14-day trend	200, full KPI + trend payload	PASS
TC-API-19	FR-18	Settings readable	GET /api/settings	200, settings object	200 — also confirms SEC-03: resendApiKey field is returned in plaintext to any authenticated caller	PASS (functionally) / finding noted
TC-API-20	FR-19	AI chat responds even on AI failure	POST /api/ai/chat	200, reply text	200 — Gemini call itself failed (404 model not found, see Section 6), rule-based fallback used , reply returned correctly	PASS
TC-API-21	FR-21	AI forecast responds even on AI failure	GET /api/ai/forecast	200, forecast object	200, deterministic fallback forecast returned correctly (Gemini again 404)	PASS
TC-API-22	FR-04	Logout clears session	POST /api/auth/logout	200, cookie cleared	200, {"success":true}	PASS
TC-API-23	FR-03	Post-logout access blocked	GET /api/items	401 after logout	401, {"error":"Unauthorized"}	PASS

Test ID	Req. ID	Test Case	Steps	Expected Result	Actual Result	Status
TC-API-24	SEC-04	Unauthenticated POST seed endpoint reachable (security finding)	POST /api/seed with no cookie	Should require auth (per NFR-01)	200, executed without authentication	FAIL — confirms SEC-04 / TD-02
TC-API-25	SEC-05	No brute-force protection on login	5 rapid wrong-passcode attempts	Should throttle/lock after N attempts	All 5 returned 401 immediately, no delay or logout	FAIL — confirms SEC-05 / TD-03
TC-API-26	SEC-07, AUTH-02	Security headers / cookie attributes	Inspect response headers	CSP/HSTS/X-Frame-Options present; cookie flags correct	No CSP/HSTS/X-Frame-Options header present (confirms SEC-07/TD-05); cookie flags HttpOnly; SameSite=lax; Max-Age=604800 correct as designed	PARTIAL {auth cookie correct; security headers absent)

5. Unit Testing

Not applicable — no unit test suite exists in the repository. A repository-wide search for `*.test.ts`, `*.spec.ts`, `__tests__`/, and Jest/Vitest/Playwright configuration files returned zero results, and no test runner is listed in `package.json`. This is recorded as technical debt TD-07 in `Technical_Debt_Plan.pdf`. The API-level tests in Section 5 provide black-box coverage of the same business logic that unit tests would normally target (validation rules, zero-floor stock rule, insufficient-stock rule) but cannot substitute for unit-level coverage of internal functions.

6. Integration Testing

The executed test sequence in Section 5 exercises several real multi-component integration paths end-to-end (HTTP layer → middleware → Zod validation → Prisma → SQLite → response), specifically:

- Login → cookie issuance → middleware-gated access to a protected route (TC-API-01–05, TC-API-22–23).
- Item creation → stock adjustment → sale recording, with quantity kept consistent across all three (TC-API-07, 10, 11, 13), confirming the Prisma `$transaction` blocks in `items/[id]/adjust/route.ts` and `sales/route.ts` behave correctly under real execution.
- AI route → Gemini client → fallback path, confirming the try/catch fallback design in `src/lib/gemini.ts` and the `ai/*` routes genuinely activates under a real failure condition (TC-API-20–21).

7. System Testing

System-level (full end-to-end, UI-inclusive) testing was **not executed** in this environment because no browser automation tooling was available. The API-level system behaviour (Sections 5–7) was verified live; the rendered front-end behaviour was not. **Not verified — execution required by student**, specifically: adding an item through the Add Item form, performing a stock adjustment through the Item Detail screen, and viewing the dashboard/reports/AI chat widget rendered in a browser.

8. User Acceptance Testing (UAT)

Not executed — no end user or student walkthrough session was performed as part of this documentation pass. **Not verified — execution required by student.** Suggested UAT scenarios for the student to run with a representative user (e.g., a shop owner) before submission:

1. Log in, add three new items, and adjust their stock without guidance from the developer.
2. Record a sale and confirm the resulting stock and revenue figures match expectations.
3. Ask the AI chat widget two natural-language questions about the current inventory.
4. Attempt to over-sell an item and confirm the error message is understandable to a non-technical user.

9. Security Testing

Executed checks: TC-API-24 (unauthenticated destructive seed endpoint — **fail**, confirms SEC-04), TC-API-25 (no login rate limiting — **fail**, confirms SEC-05), TC-API-26 (no security response headers — **fail**, confirms SEC-07; session cookie attributes correct). Verified by code review (not executed as an attack): no raw/unparameterised SQL in the codebase (SEC-01), secrets loaded from environment variables rather than hardcoded (SEC-02, with the exception of the Resend key — SEC-03), no CSRF token on state-changing routes (SEC-06). A full security assessment (e.g., dependency vulnerability scan, penetration test) was **not performed** — **not verified** — **execution required by student**. See `Technical_Debt_Plan.pdf` for remediation priorities.

10. Usability Testing

Not executed — requires a rendered browser session with a real or representative user, which was not available in this environment. **Not verified — execution required by student.**

11. Performance Testing

No formal load or stress testing was performed. However, the live test run incidentally captured real single-request latency figures from the Next.js dev server console (development mode, single concurrent user, so these are **indicative only**, not a benchmark):

Endpoint	Observed latency (dev mode, single request)
GET /api/reports/summary	1909 ms (cold)
GET /api/analytics	748 ms
GET /api/settings	636 ms
POST /api/items/[id]/adjust	71–99 ms
POST /api/sales	1180 ms
POST /api/ai/chat (Gemini call fails then fallback)	8.2 s
GET /api/ai/forecast/[itemId] (Gemini call fails then fallback)	4.6 s

The AI-route latency is notable: because the configured Gemini call currently fails (see Section 6 finding), every AI request pays the full network round-trip **and** timeout cost of the failing call before returning the fallback, rather than returning quickly. This is a real, measured effect, not an estimate, and is carried into **Technical_Debt_Plan.pdf** as a remediation item. Formal load testing (concurrent users, sustained throughput) was **not performed** — **execution required by student**.

12. Defects

Defect ID	Related Test(s)	Description	Severity	Corrective Action
Defect ID	Related Test(s)	Description	Severity	Corrective Action
DEF-01	TC-API-06, TC-API-12	Submitting invalid data that should be rejected by Zod validation (<code>quantity:-5</code> on item create; <code>quantity:0</code> on stock adjustment) returns HTTP 500 with an empty body instead of the expected 400 with a validation message. The dev-server console shows <code>unhandledRejection: Error: invalid type: boolean 'false', expected enum CodeFrameColorMode</code> , which points at a Next.js 16 development-mode error-overlay/code-frame formatting fault triggered while rendering the thrown <code>ZodError</code> , rather than the validation rule itself being wrong (the rule correctly stops the negative/zero value from reaching the database in both cases — no bad data was	Medium — user-facing error message is lost and the client receives an unhelpful 500, but no data integrity issue occurs. Must be re-tested against a production build (next build && next start) before concluding whether this is dev-mode-only.	Re-run TC-API-06/12 against a production build; if it persists, wrap the Zod <code>catch</code> block's response construction to avoid whatever value is reaching the Next.js error formatter, and add an explicit <code>instanceof ZodError</code> check before any generic error handling. Raw evidence: Supporting_Files/Test_Evidence

Defect ID	Related Test(s)	Description	Severity	Corrective Action
DEF-02	TC-API-24	POST <code>/api/seed?force=true</code> is reachable without authentication and, when called with <code>force=true</code> , deletes all inventory/sales/supplier data before reseeding. Confirmed live: the endpoint executed successfully with no session cookie.	High (data-integrity risk in a public deployment)	Remove <code>/api/seed</code> from the middleware's public whitelist, or require a one-time admin token for <code>force=true</code> . Tracked as TD-02 in <code>Technical_Debt_Plan.pdf</code> .
DEF-03	TC-API-25	No rate limiting, delay, or lockout on repeated failed login attempts; 5 rapid wrong-passcode attempts all returned immediately.	Medium	Add attempt throttling (e.g., exponential backoff or a fixed lockout window) to <code>/api/auth/login</code> . Tracked as TD-03.

Defect ID	Related Test(s)	Description	Severity	Corrective Action
DEF-04	TC-API-20, TC-API-21	The configured Gemini model (<code>gemini-1.5-flash</code>) is hardcoded in <code>src/lib/gemini.ts</code> and returns 404 Not Found from the live Google Generative AI API for this project's API key/API version — every AI request currently silently uses its fallback path instead of a genuine AI response. The application does not crash (fallback design works exactly as intended — this is a configuration/model-availability issue, not a code defect in the fallback mechanism itself).	Medium (feature works but the “AI” features are not currently AI-generated in practice)	Verify the <code>GEMINI_API_KEY</code> project has access to a currently-supported Gemini model and update the hardcoded model name in <code>src/lib/gemini.ts</code> accordingly; add a startup/health check that surfaces this condition to the developer rather than only to the request-time console log.
DEF-05	TC-API-19	<code>GET /api/settings</code> returns the <code>resendApiKey</code> field in plaintext to any authenticated caller. Not a crash or logic bug, but a secrets-handling weakness.	Medium	Move the Resend key to an environment variable, or mask it in the API response. Tracked as TD-06.

13. Regression Testing

Not applicable for this submission — this is the first documented testing pass and there is no prior baseline or automated regression suite to re-run against (see Section 8, Unit Testing). Once DEF-01 through DEF-05 are addressed, the test cases in Section 5 (particularly TC-API-06, TC-API-12, TC-API-24, TC-API-25) should be re-executed as a regression check, and the results appended to a dated addendum of this report.

14. Testing Summary

- **26 executed API-level tests:** 21 clean passes, 2 functional failures (DEF-01, appearing in TC-API-06 and TC-API-12), 2 confirmed security gaps already known from `SRS.pdf` (SEC-04 via TC-API-24, SEC-05 via TC-API-25), 1 partial result (TC-API-26 — cookie correct, headers absent).
- **0% automated test coverage** (no unit/integration/E2E suite in the repository).
- **UI, UAT, usability, and load/performance testing were not executed** in this environment and are explicitly marked as requiring student verification before submission.
- Five defects (DEF-01–DEF-05) were identified through live execution, all traceable to specific requirements in `SRS.pdf` and carried into `Technical_Debt_Plan.pdf` for prioritised remediation.

15. Requirements-to-Test Traceability

Requirement ID	Description (abridged)	Test ID(s)	Result
FR-01	Passcode authentication	TC-API-03, TC-API-04	PASS
FR-02	Session cookie issuance	TC-API-04, TC-API-26	PASS
FR-03	Route protection (middleware)	TC-API-01, TC-API-02, TC-API-23	PASS
FR-04	Logout	TC-API-22	PASS
FR-05	Dashboard/analytics	TC-API-18	PASS
FR-06	Add item	TC-API-07	PASS
FR-07	Item validation	TC-API-06	FAIL (DEF-01)
FR-08	Search/filter inventory	TC-API-05, TC-API-08	PASS
FR-09	Item detail	TC-API-05	PASS
FR-10	Edit/delete item	—	Not verified — execution required by student
FR-11	Stock adjustment	TC-API-09, TC-API-10, TC-API-11, TC-API-12	PASS / FAIL (DEF-01)
FR-12	Zero-floor stock rule	TC-API-09	PASS
FR-13	Atomic stock transaction	TC-API-10, TC-API-11	PASS
FR-14	Low-stock email (best-effort)	—	Not verified — execution required by student (requires a configured, valid Resend key)
FR-15	Record sale / oversell rejection	TC-API-13, TC-API-14	PASS
FR-16	Supplier CRUD	TC-API-15, TC-API-16	PASS
FR-17	Inventory report	TC-API-17	PASS
FR-18	Settings read	TC-API-19	PASS
FR-19	AI chat + fallback	TC-API-20	PASS
FR-20	AI description + fallback	—	Not verified — execution required by student
FR-21	AI forecast + fallback	TC-API-21	PASS

Requirement ID	Description (abridged)	Test ID(s)	Result
FR-22	AI insights + fallback	—	Not verified — execution required by student
FR-23	Auto-seed	TC-API-24	PASS (functionally) — see DEF-02 for the associated security gap
NFR-01	Auth required on protected routes	TC-API-02, TC-API-24	PASS / FAIL
NFR-04	Atomic writes	TC-API-10, TC-API-11, TC-API-13	(DEF-02 exception) PASS
NFR-05	Non-negative stock	TC-API-09	PASS
SEC-04	Unauthenticated seed gap	TC-API-24	FAIL — confirmed
SEC-05	Login rate limiting	TC-API-25	FAIL — confirmed
SEC-07	Security response headers	TC-API-26	FAIL — confirmed